

week16 예습과제

☰ 다중 선택

09 추천시스템

01. 추천 시스템

개요

- 전자상거래 업체, 콘텐츠 포털 등에서 고객의 머무르는 시간을 늘리기 위해 많이 사
- 똑똑한 추천 시스템은 많은 데이터가 추천 시스템에 축적 → 추천이 더 정확해지고 다양
- 추천 시스템 고도화 투자 ↑

온라인 스토어

- "너무 많은 상품 및 콘텐츠 + 한정된 시간"이라는 환경에서 필수 구성 요소

유형

- 콘텐츠 기반 필터링 방식
- 협업 필터링 방식
 - 최근접 이웃 협업 필터링
 - 잠재 요인 협업 필터링 ★

(하이브리드 형식도 존재)

02. 콘텐츠 기반 필터링 추천 시스템

: 사용자가 선호하는 특정한 아이템과 비슷한 콘텐츠를 가진 아이템을 추천

03. 협업 필터링

- 핵심 : 사용자가 아이템에 매긴 평점 정보나 상품 구매 이력과 같은 **사용자 행동 양식**만을 기반으로 추천

- 목표 : **축적된 사용자 행동 데이터**(사용자-아이템 평점 매트릭스)를 기반으로 사용자가 아직 평가하지 않은 아이템을 **예측 평가(Predicted Rating)**

- 사용자-아이템 평점 행렬 데이터 사용

- 레코드 레벨 형태라면 판다스의 `pivot_table()` 과 같은 함수로 행렬 형태로 변경
- 보통 희소 행렬

최근접 이웃 협업 필터링

(=메모리 협업 필터링)

- 유형
 - 사용자 기반(User-User) : 당신과 비슷한 고객들이 다음 상품도 구매했습니다
 - 행이 개별 사용자/열이 개별 아이
 - 특정 사용자와 유사한 다른 사용자 Top-N을 선정해 추천
 - 사용자 간의 유사도 측정
 - 한계 : 사용자가 평점을 매긴 아이템의 개수가 많지 않은 경우가 많아 유사도 비교가 어
 - 아이템 기반(item-item) : 이 상품을 선택한 다른 고객들은 다음 상품도 구매했습니다.

	사용자 A	사용자 B	사용자 C	사용자 D	사용자 E
상호간 유사도 높음					
다크 나이트	5	4	5	5	5
프로메테우스	5	4	4		5
스타워즈 라스트제다이	4	3	3		4

여러 사용자들의 평점을 기준으로 볼 때 '다크 나이트'와 가장 유사한 영화는 '프로메테우스'

- 행이 개별 아이템/열이 개별 사용자
- 사용자들의 아이템에 대한 호불호의 평가 척도가 유사한 아이템을 추천

⇒ 정확도 : 아이템 기반 > 사용자 기반

- 코사인 유사도 이용 ↑

잠재 요인 협업 필터링

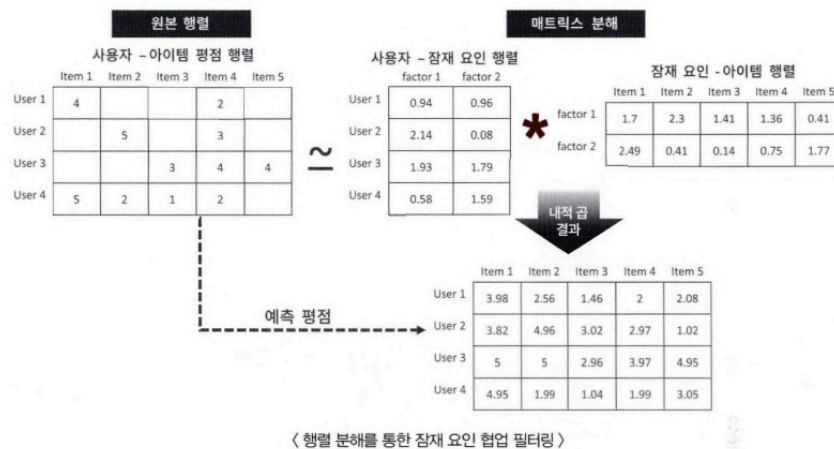
개요

→ 사용자-아이템 평점 매트릭스 속에 숨어 있는 **잠재 요인**을 기반으로 추천 예측 가능

- 행렬 분해 이용

- 행렬 분해 : 대규모 다차원 행렬을 차원 감소 기법(ex-SVD)으로 분해하는 과정에서 잠재 요인 추출

- 핵심



1. 다차원 희소행렬(사용자-아이템 행렬)을 저차원 밀집 행렬인 **사용자-잠재요인 행렬**과 **잠재요인-아이템 행렬**로 분해

2. 분해된 두 행렬의 내적을 통해 새로운 예측 사용자-아이템 평점 행렬 데이터 생성

3. 새로운 행렬 데이터로 평점X아이템의 예측 평점 생성

- 잠재요인 기반

- 영화 평점 기반 예시

- 잠재요인을 영화 장르별 선호도/영화의 장르별 요소 값이라 가정
- 아직 평점을 매기지 않은 영화를 특정 사용자의 장르 선호도 벡터와 해당 영화의 장르별 요소값 벡터를 내적함으로써 알 수 있음

행렬 분해

: 다차원 행렬을 저차원 행렬로 분해

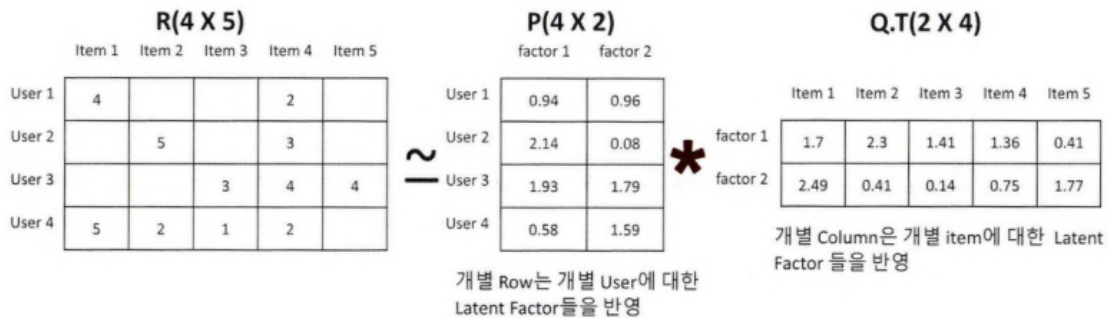
→ $M \times N$ (사용자수 $\times$ 아이템수) 행렬을 $M \times K$ (잠재요인) 행렬과 $K \times N$ 행렬로 분해

- 유형

- SVD(Singular Vector Decomposition)

- 주로 이용 but NaN 값이 없는 행렬에만 적용 가능
 - NaN 값이 많기 때문에 확률적 경사하강법(SGD)이나 ALS 이용
 - NMF(Non-Negative Matrix Factorization(인수분해))
- 미정 값 포함 모든 평점 값을 행렬 분해로 얻은 행렬들의 내적으로 계산 가능

$$R \cong \hat{R} = P * Q.T$$



확률적 경사 하강법을 이용한 행렬 분해

- 핵심 : P와 Q 행렬로 계산된 예측 R 행렬 값이 실제 R 행렬 값과 가장 최소의 오류를 가질 수 있도록 반복적인 비용 함수 최적화를 통해 P와 Q를 유추
- 절차
 1. P와 Q를 임의의 값을 가진 행렬로 설정
 2. P와 Q.T 값을 곱해 예측 R 행렬을 계산하고 예측 R 행렬과 실제 R 행렬에 해당하는 오류 값을 계산
 3. 이 오류 값을 최소화할 수 있도록 P와 Q 행렬을 적절한 값으로 각각 업데이트
 4. 만족할 만한 오류 값을 가질 때까지 2, 3번 작업을 반복하면서 P와 Q 값을 업데이트해 근사화함
- 비용 함수식과 이에 기반한 업데이트식
 - 비용 함수식

$$\min \sum (r_{u,i} - p_u q_i^t)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

- 업데이트 식

$$\begin{aligned} p'_u &= p_u + \eta (e_{(u,i)} * q_i - \lambda * p_u) \\ q'_i &= q_i + \eta (e_{(u,i)} * p_u - \lambda * q_i) \end{aligned}$$

◦ 기호 의미

- p_u : P 행렬의 사용자 u행 벡터
- q_i^t : Q 행렬의 아이템 i행의 전치 벡터(transpose vector)
- $r_{u,i}$: 실제 R 행렬의 u행, i열에 위치한 값.
- $\hat{r}_{u,i}$: 예측 R 행렬의 u행, i열에 위치한 값. $p_u * q_i^t$ 로 계산
- $e_{(u,i)}$: u행, i열에 위치한 실제 행렬 값과 예측 행렬 값의 차이 오류. $r_{u,i} - \hat{r}_{u,i}$ 로 계산
- η : SGD 학습률
- λ : L2 규제(Regularization) 계수